Entwicklung eines Sprachlern-Chatbots

Andreleene Prado und Ronja Rüther
Fachhochschule Südwestfalen

Schriftliche Ausarbeitung im Modul
„Natural Language Processing“

10. März 2026

Inhaltsverzeichnis

1. Einleitung **3**
1.1. Hintergrund (Ronja Rüther) 3
1.2. Stand der Technik (Andreleene Prado) 3
1.3. Zielsetzung (Ronja Rüther) 4

2. Simulationsumgebung und Trainingsdaten (Andreleene Prado) **5**

3. Architektur (Ronja Rüther) **5**
3.1. Systemarchitektur (Ronja Rüther) 5
3.2. Satzmodus auf Wortebene (Ronja Rüther) 6
3.2.1. Implementierung der Satzgenerierung (Ronja Rüther) 6
3.2.2. Hybride Evaluation und Feedback-Logik (Ronja Rüther) 7
3.3. Satzmodus auf Satzebene (Andreleene Prado) 8
3.4. Konversationsmodus (Andreleene Prado) 10

4. Evaluation der Ergebnisse **11**

5. Kritische Reflexion und Fazit **11**
5.1. Aufgetretene Probleme 11
5.1.1. Zu komplexe Satzstrukturen bei hohem Schwierigkeitsgrad (Ronja Rüther) 11
5.1.2. Problem bei der Erkennung konjugierter Wortformen (Ronja Rüther) . . 12
5.1.3. Fehlerhafte Erkennung von Satzzeichen (Ronja Rüther) 12
5.2. Vergleich zum Stand der Technik 12
5.3. Fazit und Ausblick 12

1. Einleitung

1.1. Hintergrund (Ronja Rüther)

In den letzten Jahren sind Chatbots, die auf Large Language Models (LLMs) basieren, wie etwa ChatGPT oder Google Gemini, immer beliebter geworden und verzeichnen stetig wachsende Nutzerzahlen. Für viele Personen sind Chatbots zu einem festen Bestandteil im Alltag geworden und werden herkömmlichen Suchmaschinen bei der Informationsbeschaffung oft vorgezogen. Der Hauptvorteil liegt darin, dass Nutzer schneller eine direkte Antwort auf spezifische Fragestellungen erhalten und die eigenständige Suche nach Information auf verschiedenen Webseiten dadurch weitgehend entfällt. [1]

Diese technologische Entwicklung hat auch im Bereich der Bildung große Veränderungen mit sich gebracht. Während traditionelle Sprachlern-Apps oft auf statischen Übungen und vorgegebenen Antwortmöglichkeiten basieren, ermöglichen LLMs die dynamische Generierung von Lerninhalten. Da diese Modelle die natürliche Sprache nicht nur erkennen, sondern ebenfalls die Rechtschreibung und Grammatik bewerten können, können sie die Rolle eines digitalen Sprachlehrers übernehmen. [2] Dabei erhält der Nutzer individuelles Feedback auf seine Eingaben und kann gezielt auf typische Fehlerquellen hingewiesen werden. Da diese Interaktion zwischen dem Nutzer und der Künstlichen Intelligenz (KI) in Echtzeit erfolgt, erhält der Nutzer das Gefühl, als würde er mit einer realen Person digital kommunizieren. [2]

Trotz dieser Fortschritte gibt es immer noch Herausforderungen, die diese Modelle mit sich bringen. [2] Dazu zählen beispielsweise die individuelle Anpassung der Satzgenerierung an das Sprachniveau des Nutzers sowie die Generierung von hilfreichem und nachvollziehbarem Feedback.

1.2. Stand der Technik (Andreleene Prado)

Mit der fortgeschrittenen Entwicklung der Künstlichen Intelligenz sowie der zunehmenden Verfügbarkeit von Open-Source-Modellen wie LLaMa gibt es heute deutlich mehr Möglichkeiten, Chatbots für spezifische Aufgaben zu entwicklen und anzupassen. Dadurch können auch Personen oder kleinere Entwicklerteams eigene Chatbots erstellen, die auf bestimmte Anwendungsbereiche zugeschnitten sind.

Ein wichtige Anwendungsbereich ist das Sprachlernen. Chatbots können dabei als interaktive Gesprächspartner dienen und Lernenden ermöglichen, Sprachkenntnisse durch Dialoge zu üben. Eine der bekanntesten Sprachlern-Apps ist **Duolingo**. Die Plattform nutzt unter anderem LLMs, um neue Übungen und Aufgaben zu generieren sowie personalisierte Lerninhalte für Nutzer bereitzustellen.[3]

Das KI-Modell **Birdbrain** von Duolingo wird unter anderem zur automatischen Erstellung von Lernübungen eingesetzt. Dabei werden LLMs durch strukturierte Prompts gesteuert, die bestimmten pädagogische und linguistische Vorgaben enthalten. Der Prozess zur Erstellung neuer Übungen besteht aus mehreren Schritten. Zunächst werden die Inhalte einer Lektion geplant. Anschließend wird das KI vorbereitet, indem relevante Parameter in den Prompt intergriert werden, wie die Zielsprache, Sprachniveau, sowie das Thema der Lektion. Die generierten Aufgabe können von den Entwicklern überprüft und bei Bedarf angepasst werden.[3]

Eine neuere Funktion der Plattform ist **Duolingo Max**, ein erweitertes Lernangebot, das auf LLMs basiert. Dabei werden moderne KI-Modelle eingesetzt, um zusätzliche interaktive Lernmöglichkeiten bereitzustellen. Mit Duolingo Max können Nutzer ihre Sprachkenntnisse in realistischeren Kommunikationssituationen üben. Dazu gehören beispielweise simulierte Gespräche in Form von Rollenspielen oder Video-Anrufe mit einem virtuellen Gesprächspartner. Die Funktion basiert auf dem Sprachmodell GPT-4, das für die Generierung von Antworten und Feedback verwendet wird. Dadurch kann das System auf Eingaben der Lernenden flexibel reagieren und kontextabhängige Antworten erzeugen. Zusätzlich ermöglicht das Modell eine höhere Genauigkeit sowie schnellere Generierung von Rückmeldungen. [4]

Der Einsatz von KI ermöglicht es, Lerninhalte deutlich schneller zu produzieren als mit rein manuellen Prozessen. Da Plattformen wie Duolingo eine sehr große Anzahl an Nutzern bedienen, erleichtert diese Automatisierung die kontinuierliche Erweiterung und Aktualisierung von Kursinhalten.

1.3. Zielsetzung (Ronja Rüther)

Das Ziel dieses Projekts ist die Entwicklung einer webbasierten Sprachlern-Applikation namens „Lingotrain AI“, die die Möglichkeiten von Large Language Models nutzt, um ein individuelles Sprachtraining zu ermöglichen. Die Anwendung unterstützt die Zielsprachen Englisch, Französisch und Spanisch, wobei Deutsch immer als Ausgangssprache dient. Die Schwerpunkte der Entwicklung liegen dabei auf der dynamischen Satzgenerierung und der Evaluation der Nutzereingaben.

Ein zentraler Aspekt ist die Implementierung einer Logik, die zu einer gegebenen Vokabel dynamisch Beispielsätze generiert. Dabei ist besonders darauf zu achten, dass die Sätze inhaltlich passend und grammatikalisch korrekt sind. Außerdem wird ein Schwierigkeitsgrad gegeben (einfach, mittel, schwer), an den der Satz angepasst werden soll. Dieser basiert auf der Anzahl, wie oft der Nutzer die gegebene Vokabel bereits gelernt hat.

Des Weiteren soll das System die Antwort des Nutzers evaluieren. Um die Lernmotivation zu steigern, ist ein zweistufiges Feedback-System geplant. Dabei soll der Nutzer bei falschen Eingaben zunächst einen hilfreichen Hinweis erhalten, um einen zweiten Lösungsversuch zu ermöglichen. Anschließend soll ihm detailliert Feedback gegeben werden, in dem ein Korrekturvorschlag und eine Erklärung des Fehlers enthalten ist. Auch bei korrekten Antworten wird eine Bestätigung mit kurzem Feedback ausgegeben.

Zusätzlich zum Training mit einzelnen Sätzen ist auch ein Konversationsmodus vorgesehen. In diesem Modus soll eine vorgegebene Vokabel aktiv in die Konversation eingebunden werden. Das Modell generiert hierfür passende Sätze, die für eine Konversation mit dem Nutzer geeignet sind. Auf diese Sätze muss der Nutzer reagieren, sodass ein interaktives Gespräch entsteht. Diese Konversation findet statt, bis der Nutzer das Zielwort mehrfach erfolgreich angewendet hat oder eine maximale Anzahl an Interaktionen erreicht ist. Abschließend analysiert das Modell die Konversation und erstellt detailliertes Feedback, um dem Nutzer Verbesserungen sowie häufige Fehlerquellen nennen zu können.

2. Simulationsumgebung und Trainingsdaten (Andreleene Prado)

Die Implementierung des Systems erfolgte in der Entwicklungsumgebung Visual Studio Code unter Verwendung der Programmiersprache Python. Vor der Integration des Backends mit dem Frontend der Web-UI wurden die einzelnen Funktionen zunächst über das Terminal getestet, um die korrekte Verarbeitung der Eingaben und die Funktionsweise der NLP-Komponenten zu überprüfen.

Für den Satzgenerierungsmodus wurden verschiedene Textausgaben direkt im Terminal getestet. Dabei wurde eine Liste von Testfällen erstellt, die englische, französische und spanische Sätze umfasst. Zu jedem Testfall wurden sowohl der originale Referenzsatz als auch die vom Benutzer eingegebene Variante sowie die erwartete Auswertung dokumentiert. Diese Testfälle dienten dazu, die korrekte Verarbeitung und Bewertung der Benutzereingaben zu überprüfen.

Für den Konversationsmodus wurde eine einfache textbasierte Mini-Konversation im Terminal implementiert. Diese basierte auf einem festgelegten Vokabel sowie den entsprechenden englischen Übersetzungen, um den Ablauf der Interaktion und die Reaktionslogik des Systems zu testen. Dadurch konnte überprüft werden, ob der Chatbot auf Benutzereingaben korrekt reagiert und der Dialogfluss wie vorgesehen funktioniert.

In diesem Projekt wurden keine eigenen Trainingsdatensätze erstellt und es wurden keine Modelle von Grund auf trainiert. Stattdessen wurden vortrainierte NLP-Modelle verwendet, die bereits auf großen Textkorpora trainiert wurden.

Zusätzlich wurde ein Large Language Model (LLM) eingesetzt, um die Generierung von Antworten zu unterstützen. Dieses Modell basiert auf umfangreichen Trainingsdaten aus unterschiedlichen Textquellen und ist in der Lage, kontextabhängige sowie sprachlich kohärente Antworten zu erzeugen.

3. Architektur (Ronja Rüther)

In diesem Kapitel wird die Architektur der Anwendung beschrieben. Dabei werden zunächst die grundlegende Systemarchitektur sowie die Kommunikation zwischen Frontend und Backend dargestellt. Anschließend erfolgt eine detailliertere Erläuterung der Implementierung des Satzmodus sowie Konversationsmodus.

3.1. Systemarchitektur (Ronja Rüther)

Die Architektur von „Lingotrain AI“ basiert auf einem modernen **Client-Server-Modell**. Das Frontend wurde in *Vue.js* implementiert und ist für die Benutzerinteraktion sowie die Darstellung der Inhalte verantwortlich. Aus dem Frontend werden alle Eingaben des Nutzers an das Backend weitergeleitet, welches mit *FastAPI* in Python umgesetzt ist. Das Backend übernimmt die Verarbeitung der Nutzereingaben, steuert die Anbindung an Large Language Models (LLMs) und übernimmt verschiedene Aufgaben der natürlichen Sprachverarbeitung (NLP).

Die Kommunikation zwischen dem Frontend und dem Backend erfolgt über definierte *REST-API*-Endpunkte. Die zentralen Endpunkte der Applikation sind folgende:

- `/generate-sentence`: Generiert einen Beispielsatz zu einer vorgegebenen Vokabel.

- `/evaluate-answer`: Bewertet eine Nutzerantwort im Satzmodus auf Wortebene, d.h. die Vokabel besteht nur aus einem Wort. Es wird zudem auch Feedback und ein Korrekturhinweis geliefert.
- `/evaluate-sentence`: Bewertet eine Nutzerantwort im Satzmodus auf Satzebene und liefert ebenfalls Feedback.
- `/conversation/start`: Startet den Konversationsmodus, wobei der erste Satz für die Interaktion mit dem Nutzer generiert wird.
- `/conversation/next`: Führt die Konversation fort und erstellt weitere Sätze für den interaktiven Dialog.

Für die Satzgenerierung und die Bewertung werden LLMs verwendet. Dabei wird ein Prompt erstellt, der die Zielvokabel, die Zielsprache und den Schwierigkeitsgrad berücksichtigt. Neben der LLM-basierten Bewertung kommen NLP-Tools wie *SpaCy* für Tokenisierung, Lemmatisierung und POS-Tagging zum Einsatz. Außerdem wird *Sentence-BERT* zur Berechnung der semantischen Ähnlichkeit zwischen der Nutzereingabe und dem Referenzsatz genutzt.

Bevor die Eingaben an die LLMs gesendet werden, prüft das Backend die Sätze auf Plausibilität und Vollständigkeit. Die Ergebnisse dieser Analysen werden in Echtzeit über die API an das Frontend zurückgegeben, sodass der Nutzer das Feedback direkt erhält. Durch diese Trennung von Frontend und Backend werden die Verantwortlichkeiten klar abgegrenzt.

3.2. Satzmodus auf Wortebene (Ronja Rüther)

Im Satzmodus auf Wortebene wird der Fokus darauf gelegt, dass der Nutzer einen generierten Satz zu einer Zielvokabel korrekt in die Zielsprache übersetzen kann. Die Implementierung erfolgt vollständig im Backend und nutzt die bereits beschriebenen NLP- und LLM-Komponenten. Die generative KI wird daher mit den klassischen NLP-Methoden kombiniert, um eine präzise Bewertung zu gewährleisten.

3.2.1. Implementierung der Satzgenerierung (Ronja Rüther)

Bei der Satzgenerierung wird nicht einfach auf eine statische Datenbank mit festen Beispielsätzen zurückgegriffen, sondern die Sätze werden zur Laufzeit zu einer gegebenen Zielvokabel über die Funktion `generate_sentence(...)` erzeugt. Dadurch wird der Lerninhalt variiert und der Nutzer kann immer wieder unterschiedliche Sätze zu der gleichen Vokabel lernen.

Damit die Kommunikation mit verschiedenen Sprachmodellen modellunabhängig möglich ist, wird die Bibliothek **liteLLM** als Abstraktionsschicht eingesetzt. Diese bildet die Brücke zwischen unserem FastAPI-Backend und dem Sprachmodell GPT-4o. Den größten Teil der Satzgenerierung nimmt die Promptgenerierung und damit das Design des `sentence_prompt(...)` ein. Dieser Prompt steuert das Verhalten des verwendeten LLMs, sodass dem Modell strikte Regeln für verschiedene Schwierigkeitsgrade erteilt werden:

- **Einfach:** Der Fokus liegt auf Hauptsätzen im Präsens ohne Adjektive.
- **Mittel:** Es werden Nebensätze eingeführt. Das können z.B. Kausalsätze mit „weil“ sein.
- **Schwer:** Der Satz besteht aus komplexeren Strukturen mit variablen Zeitformen.

Ein entscheidender Aspekt des Prompt Designs ist außerdem die Verwendung von *Negative Constraints*. Damit der generierte Satz im Backend weiterverarbeitet werden kann, wird das Modell angewiesen, keine einleitenden Sätze, Erklärungen oder Übersetzungen auszugeben. Dadurch wird sichergestellt, dass die Antwort des Modells direkt als String-Variable verwendet werden kann, ohne dass eine aufwendige Bereinigung des Textes notwendig ist.

Um eine hohe Variabilität der Lerninhalte sicherzustellen und zu verhindern, dass der Nutzer bei der gleichen Vokabel immer denselben Satz erhält, wurde das Modell unter Verwendung drei Parameter konfiguriert:

1. **n = 5:** Es werden intern fünf verschiedene Satzvarianten generiert. Aus dieser Auswahl wird dann mittels einer Zufallsfunktion ein Satz extrahiert und an das Frontend übermittelt.
2. **temperature = 0.9:** Durch diesen Wert wird die Kreativität des Modells gesteigert. Ein höherer Wert sorgt dafür, dass die Wortfolgen weniger vorhersehbar gewählt werden.
3. **top_p = 0.9:** Das sogenannte *Nucleus Sampling* ist dafür zuständig, dass die grammatikalische Stabilität trotz der hohen Kreativität des Modells erhalten bleibt. Dafür wird die Wortauswahl auf die wahrscheinlichsten 90% der Wortkandidaten beschränkt und dadurch verhindert, dass das Modell zu abwegige Begriffe verwendet.

Durch die Kombination dieser Parameter wird ein guter Mittelweg für die didaktische Korrektheit und die Abwechslung in den Sätzen erzielt.

3.2.2. Hybride Evaluation und Feedback-Logik (Ronja Rüther)

Die größte Herausforderung des Satzmodus stellt die Bewertung der Nutzereingaben dar. Da die Nutzer die Zielwörter oft konjugieren oder leicht vom Original abweichen, würde ein einfacher String-Vergleich zu einer hohen Fehlerrate führen. Um die Genauigkeit der Korrektur zu erhöhen und sich nicht nur auf das LLM zu verlassen, aber auch um Kosten zu sparen und schnellere Antworten liefern zu können, wurde die Bewertung in zwei Stufen aufgeteilt. Bei dieser hybriden Evaluation gibt es zunächst eine regelbasierte Vorprüfung und anschließend eine semantische Analyse mittels LLM.

Regelbasierte Vorprüfung mittels SpaCy:
Zunächst wird die Antwort des Nutzers lokal im Backend analysiert, bevor ein Aufruf des LLMs erfolgt. Dies hat den Zweck, Kosten zu sparen, da jeder LLM-Aufruf Kosten verursacht. Bei der Vorprüfung im Backend wird die NLP-Bibliothek *SpaCy* verwendet.

Den ersten Schritt bei dieser regelbasierten Vorprüfung bildet die **Lemmatisierung**. Die Funktion `contains_target_word(...)` reduziert dabei die Zielvokabel und den eingegebenen Satz des Nutzers auf ihre Grundformen (Lemmata). Dadurch wird sichergestellt, dass beispielsweise die Eingabe „went“ als korrekte Verwendung der Vokabel „go“ erkannt wird.

Damit auch Mehrwort-Vokabeln, wie z.B. „ice cream“, erkannt und identifiziert werden können, prüft ein Algorithmus die Wortgruppen innerhalb eines Satzes. Dadurch wird verhindert, dass korrekte Antworten aufgrund von Leerzeichen in der Eingabe fälschlicherweise als falsch bewertet werden.

Da ein eingegebener Satz auch mit einem Satzzeichen enden soll, wird vorab mithilfe der Funktion `check_sentence_end(...)` geprüft, ob das letzte Zeichen im Satz ein Punkt, Ausrufezeichen oder

Fragezeichen enthält. Nachfolgende Leerzeichen werden dabei durch Verwendung der `.strip()`-Funktion ignoriert.

Zuletzt wird durch das *POS-Tagging* (Part-of-Speech) geprüft, ob die Eingabe des Nutzers ein Verb enthält und somit einen vollständigen Satz darstellt. Dazu wurde die Funktion `is_sentence_plausible(...)` implementiert, die linguistische Merkmale von *SpaCy* nutzt, um die Verben und Hilfsverben zu identifizieren. Dabei wurde ein zweistufiger Check implementiert, der verhindert, dass bloße Wortlisten oder unvollständige Sätze an das LLM weitergeleitet werden:

1. **POS-Tagging und Morphologie:** Das System prüft jedes Token auf sein POS-Tag (`VERB`) oder (`AUX`). Zusätzlich werden morphologische Merkmale wie Zeitform (*Tense*) oder Personalform (*Person*) abgefragt. Dies ist für die Sprachen Französisch und Spanisch besonders wichtig, um komplex konjugierte Verben nicht zu übersehen.
2. **Dependenz-Analyse:** Es wird zusätzlich geprüft, ob ein Wort als Wurzel (`dep_ == „ROOT"`), also zentrales Element des Satzes, gilt, von dem alle anderen Wörter abhängen. Dadurch können auch Sätze korrekt erkannt werden, bei denen die automatische Wortart-Erkennung unsicher ist.

Semantische Analyse und Feedback:
Sind die regelbasierten Vorprüfungen ohne Fehler durchlaufen, wird das LLM zur finalen semantischen Prüfung verwendet. Das Modell kann dabei durch sein Sprachverständnis beurteilen, ob die Eingabe des Nutzers die Bedeutung des Referenzsatzes korrekt wiedergibt. Im `evaluation_prompt(...)` wird das Modell dabei angewiesen, die Bedeutung der Nutzerantwort mit dem Referenzsatz zu vergleichen.

Auf Basis der Bewertung soll das Modell auch Feedback für den Nutzer generieren. Um die Frustration bei kleineren Fehlern zu minimieren, wird ein zweistufiger Ansatz verfolgt. Wenn das Modell ein fehlerhaftes Satzzeichen oder eine minimale Abweichung erkennt, vergibt das Modell ein Rating von `almost_correct`. Der Nutzer erhält in diesem Fall einen Tipp (Hint), ohne dass sofort die vollständige Lösung eingeblendet wird. Auch bei ganz falschen Antworten (`rating = „wrong"`) wird so vorgegangen. Erst im zweiten Versuch wird eine korrekte Antwort sowie eine technische Begründung des Fehlers geliefert.

Um die Weiterverarbeitung im Backend sicherzustellen, wird das Modell angewiesen, die Antwort als JSON-Objekt zurückzugeben. Dieses Objekt enthält dann die Felder für das `rating`, einen Korrekturvorschlag (`corrected_sentence`), einen Hinweis (`hint`) sowie ein kurzes Feedback (`short_feedback`) und die technische Begründung (`comment`). Für die Speicherung des Ergebnisses in der Datenbank wird außerdem ein Wahrheitswert erwartet (`semantically_correct`). Durch diese Strukturierung kann die logische Bewertung von der textuellen Rückmeldung an den Nutzer klar getrennt werden.

3.3. Satzmodus auf Satzebene (Andreleene Prado)

Da die Vokabelliste nicht nur einzelne Wörter, sondern auch vollständige Sätze enthält, müssen diese ebenfalls bei der Evaluation berücksichtigt werden. Im Frontend werden vollständige Sätze jedoch nicht an das Backend zur Satzgenerierung gesendet, sondern direkt ausgegeben. Der Grund dafür ist, dass der Satz bereits vollständig vorliegt und eine erneute Generierung durch ein

Large Language Model (LLM) keinen zusätzlichen Nutzen bringen würde. Dies würde lediglich zu einer unnötigen Nutzung von Rechenressourcen führen.

Für die Bewertung von Sätzen wird ein eigener REST-API-Endpunkt verwendet, der sich von dem Endpunkt für einzelne Wörter unterscheidet. Die Auswertung erfolgt in der Funktion `evaluate_sentence_similarity`. Als Eingabeparameter erhält diese Funktion den vom Benutzer eingegebenen Satz sowie den Referenzsatz, der als korrekte Übersetzung in der Datenbank gespeichert ist. Die Parameter `target_word` und `target_language` sind für den Satzmodus nicht erforderlich. Sie sind jedoch weiterhin Bestandteil des `EvalRequest`-Modells, da dieses ursprünglich für die Evaluation einzelner Wörter entwickelt wurde.

Zunächst werden sowohl der eingegebene Satz als auch der Referenzsatz normalisiert. Dazu wird die Hilfsfunktion `normalize()` verwendet. In diesem Schritt wird der Text in Kleinbuchstaben umgewandelt, führende und nachfolgende Leerzeichen werden entfernt und Satzzeichen werden gelöscht. Diese Normalisierung erleichtert den Vergleich der beiden Sätze, da Unterschiede in Groß- und Kleinschreibung oder Interpunktion keinen Einfluss auf die Bewertung haben.

Nach der Normalisierung wird zunächst geprüft, ob der eingegebene Satz exakt mit dem Referenzsatz übereinstimmt. Ist dies der Fall, wird die Antwort als korrekt bewertet. Diese Vorgehensweise setzt voraus, dass die in der Datenbank gespeicherten Sätze inhaltlich sowie grammatikalisch korrekt sind. Wenn der Benutzer den Satz exakt reproduziert, wird dieser als korrekt erkannt und entsprechend als gelernt markiert.

Falls keine exakte Übereinstimmung vorliegt, wird die semantische Ähnlichkeit zwischen beiden Sätzen berechnet. Ziel ist es zu überprüfen, ob der vom Benutzer eingegebene Satz denselben Sinn wie der Referenzsatz ausdrückt, auch wenn die Formulierung leicht abweicht.

Für diese Berechnung wird ein Sentence-BERT-Modell (SBERT) verwendet. SBERT basiert auf der BERT-Architektur und ist speziell dafür optimiert, semantische Ähnlichkeiten zwischen Sätzen zu bestimmen. Dabei werden beide Sätze zunächst in sogenannte Embeddings umgewandelt, also in numerische Vektorrepräsentationen im semantischen Vektorraum. Anschließend wird die Cosinus-Ähnlichkeit zwischen diesen Vektoren berechnet.

Der resultierende Ähnlichkeitswert liegt zwischen 0 und 1, wobei höhere Werte eine stärkere semantische Übereinstimmung anzeigen. Abhängig vom berechneten Wert wird die Benutzereingabe in verschiedene Kategorien eingeteilt:

- **Ähnlichkeit** ≥ 0.85**:** Der Satz ist sinngleich mit dem Referenzsatz und wird als nahezu korrekt bewertet.
- **Ähnlichkeit** ≥ 0.65**:** Der Satz ist teilweise korrekt, enthält jedoch kleinere Abweichungen.
- **Ähnlichkeit** < 0.65**:** Der Satz wird als falsch bewertet.

Zusätzlich wird dem Benutzer Feedback gegeben, das den Ähnlichkeitswert sowie einen Verbesserungsvorschlag enthält. Als Vorschlag wird dabei der korrekte Referenzsatz aus der Datenbank angezeigt, sodass der Benutzer seine Eingabe direkt mit der korrekten Formulierung vergleichen kann.

3.4. Konversationsmodus (Andreleene Prado)

Im Konversationsmodus werden alle Wörter eines Lernsets berücksichtigt. Zu Beginn einer neuen Konversation wird im Frontend zufällig eine Vokabel aus der ausgewählten Lernsetliste ausgewählt. Diese wird anschließend zusammen mit der Zielsprache und der Übersetzung über einen REST-API-Endpunkt an das Backend gesendet.

Die Konversation wird durch die Funktion `start_conversation()` gestartet. Die wichtigsten Parameter sind das Zielwort (`target_word`), die Zielsprache (`target_language`) sowie die Übersetzung (`translation`). Diese Informationen werden verwendet, um einen geeigneten Prompt für das Large Language Model (LLM) zu erstellen. Die Übersetzung des Wortes ist dabei wichtig, damit das Modell den semantischen Kontext des Wortes korrekt interpretieren kann.

Zusätzlich enthält der Prompt bestimmte Regeln, die sich an der Wortart des Fokuswortes orientieren. Dadurch wird sichergestellt, dass der generierte Dialog den Lernenden dazu anregt, das Zielwort aktiv zu verwenden.

Wortartabhängige Regeln:

- **Verb:** Der Assistent stellt Fragen zu Handlungen oder Aktivitäten und fragt nach persönlichen Erfahrungen oder Gewohnheiten, z. B. „Wann machst du das?“ oder „Warum machst du das?“
- **Nomen:** Der Assistent stellt Fragen, bei denen der Lernende das Nomen natürlich als Objekt verwenden kann, z. B. durch Fragen nach Besitz, Meinungen oder Erfahrungen.
- **Adjektiv:** Der Assistent stellt Fragen zu Meinungen oder Beschreibungen, z. B. „Findest du das auch so?“ oder „Wann ist etwas so?“

Das LLM generiert anschließend die erste Frage der Konversation, welche an das Frontend zurückgegeben und dort im Chat angezeigt wird.

Wenn der Benutzer antwortet, wird die Funktion `next_turn()` aufgerufen. Diese erhält den aktuellen Konversationszustand (`state`), die Benutzereingabe (`user_input`) sowie die Zielsprache als Parameter.

Zunächst wird die Benutzereingabe mithilfe eines spaCy-Sprachmodells analysiert. Dabei wird überprüft, ob die Eingabe einen plausiblen vollständigen Satz enthält. Falls kein Verb vorhanden ist oder die Eingabe nicht als vollständiger Satz interpretiert werden kann, wird der Benutzer aufgefordert, einen vollständigen Satz zu formulieren.

Anschließend wird geprüft, ob die Antwort inhaltlich zur letzten Frage des Assistenten passt. Dazu wird die letzte Assistentenfrage aus dem Chatverlauf extrahiert und zusammen mit der Benutzereingabe an das LLM übergeben. Das Modell bewertet anhand eines speziellen Prompts, ob die Antwort relevant ist. Diese Prüfung verhindert, dass der Benutzer lediglich einen beliebigen Satz mit dem Zielwort schreibt, ohne auf die gestellte Frage einzugehen.

Wenn beide Prüfungen erfolgreich sind, wird die Benutzereingabe akzeptiert und im Konversationszustand gespeichert. Anschließend wird analysiert, wie häufig das Zielwort im bisherigen Gespräch verwendet wurde. Hierfür wird der Chatverlauf mit einem spaCy-Modell verarbeitet und mithilfe der Funktion `contains_target_word()` geprüft, ob das Zielwort in den Benutzernachrichten vorkommt.

Die Anzahl der Wortverwendungen (`usage_count`) sowie die Anzahl der bisherigen Gesprächsrunden (`turn_count`) werden im Konversationszustand gespeichert und kontinuierlich aktualisiert.

Die Konversation endet unter zwei möglichen Bedingungen:

- Wenn das Zielwort mindestens zweimal vom Benutzer verwendet wurde und der Benutzer mindestens drei Antworten im Dialog gegeben hat. In diesem Fall wird davon ausgegangen, dass das Wort ausreichend geübt wurde.
- Wenn der Benutzer zehn Gesprächsrunden erreicht hat, ohne die erste Bedingung zu erfüllen. In diesem Fall wird die Konversation beendet, um zu vermeiden, dass der Dialog unnötig lange fortgesetzt wird.

Nach Beendigung der Konversation wird der gesamte Chatverlauf vom LLM bewertet. Hierzu wird die Funktion `evaluate_conversation()` verwendet, die den Verlauf an das LLM übergibt und ein strukturiertes Feedback generieren lässt. Dieses Feedback enthält eine Bewertung der Konversation, einen kurzen Kommentar zur Sprachverwendung sowie gegebenenfalls Hinweise zur Verbesserung.

Um die Kosten der LLM-Anfragen zu reduzieren und gleichzeitig den relevanten Kontext zu erhalten, wird der Chatverlauf vor jeder Anfrage gekürzt. Dabei bleiben die Systemnachricht sowie maximal fünf der letzten Nachrichten im Verlauf erhalten. Diese Begrenzung stellt sicher, dass das Modell ausreichend Kontext erhält, ohne unnötig viele Tokens zu verarbeiten.

Nach Abschluss einer Konversation wird im Frontend automatisch eine neue Konversation mit einer anderen zufällig ausgewählten Vokabel gestartet. Die zuvor verwendete Vokabel wird aus der Liste entfernt, sodass sie nicht erneut in derselben Lernsession verwendet wird.

4. Evaluation der Ergebnisse

5. Kritische Reflexion und Fazit

5.1. Aufgetretene Probleme

5.1.1. Zu komplexe Satzstrukturen bei hohem Schwierigkeitsgrad (Ronja Rüther)

Eine Herausforderung bei der Satzgenerierung bestand in der Anpassung der Sätze an den Schwierigkeitsgrad. Besonders beim Schwierigkeitsgrad „schwer“ erzeugte das Sprachmodell teilweise sehr lange und stark verschachtelte Sätze. Diese enthielten häufig mehrere Nebensätze und verfolgten einen sehr poetischen Stil mit schweren zusätzlichen Wörtern, wodurch die Übungen für den Nutzer unnötig schwierig wurden.

Um dieses Problem zu beheben, wurde das Prompt-Design mehrfach überarbeitet und dem Modell präzisere Anweisungen gegeben, um die Satzlänge zu begrenzen und die Struktur der Sätze zu begrenzen. Zusätzlich wurde der Fokus auch mehr auf die klare und verständliche Generierung des Satzes gelegt.

Durch diese Anpassung konnte erreicht werden, dass im Schwierigkeitsgrad „schwer“ die Sätze anspruchsvoller werden, aber dennoch verständlich und für einen Lernenden lösbar bleiben.

5.1.2. Problem bei der Erkennung konjugierter Wortformen (Ronja Rüther)

Bei der Evaluation der Nutzereingaben trat ein weiteres Problem bei der Erkennung bestimmter Wortformen auf. Besonders im Spanischen wurden korrekt verwendete Wörter teilweise als falsch bewertet. Dazu zählte beispielsweise das Wort „como“, welches eine konjugierte Form des Verbs „comer“ darstellt.

Zu Beginn wurde bei der Überprüfung, ob die Zielvokabel im Satz vorkommt, hauptsächlich ein direkter Vergleich der eingegebenen Wörter mit der Zielvokabel durchgeführt. Dadurch wurden konjugierte Formen nicht zuverlässig erkannt.

Zur Lösung dieses Problems wurde die Evaluation um eine weitere Analyse ergänzt, die *SpaCy* nutzt, um Lemmatisierung durchzuführen und das Wort auf die Grundform zu reduzieren. Wie bereits im Anschnitt 3.2.1 beschrieben, wird dabei nun auch insbesondere das Wurzelelement des Satzes (`ROOT`) berücksichtigt. Dadurch lassen sich auch in der spanischen Sprache die konjugierten Formen zuverlässiger erkennen und bewerten.

5.1.3. Fehlerhafte Erkennung von Satzzeichen (Ronja Rüther)

Ein weiteres Problem trat bei der Bewertung der Satzzeichen auf. Zu Beginn bewertete das Sprachmodell Sätze häufig als fehlerhaft (`rating = „almost_correct"`), da das Satzzeichen fehlen würde, obwohl die Nutzereingabe mit dem korrekten Satzzeichen endete.

Die Ursache bestand darin, dass die Bewertung der Satzzeichen vollständig durch das Sprachmodell erfolgte. In einigen Fällen hat dabei das Modell das vorhandene Satzzeichen allerdings ignoriert.

Um dieses Problem zu lösen, wurde eine zusätzliche regelbasierte Vorprüfung im Backend implementiert. Die Funktion `check_sentence_end(...)` prüft vor dem LLM-Aufruf, ob der Satz mit einem gültigen Satzzeichen endet. Ist das der Fall, prüft das LLM anschließend nur noch, ob das verwendete Satzzeichen im Kontext des Satzes passend ist, aber nicht mehr, ob überhaupt ein Satzzeichen vorhanden ist.

5.2. Vergleich zum Stand der Technik

5.3. Fazit und Ausblick

Literatur

[1] A. Mühlbauer. „ChatGPT Statistiken (2025),“ besucht am 5. März 2026. Adresse: https://www.aipunk.de/chatgpt-statistiken-2025/

[2] M. Pullen und H. Hamilton. „Vorteile der künstlichen Intelligenz in der Bildung,“ besucht am 5. März 2026. Adresse: https://www.jamf.com/de/blog/vorteile-ki-in-Bildung/

[3] P. Henry. „How Duolingo uses AI to create lessons faster,“ besucht am 8. März 2026. Adresse: https://blog.duolingo.com/large-language-model-duolingo-lessons/

[4] D. Team. „Introducing Duolingo Max, a learning experience powered by GPT-4,“ besucht am 8. März 2026. Adresse: https://blog.duolingo.com/duolingo-max/